

Docker、Docker-compose 入门教程

#docker #容器 #教学 #docker-compose

飞致云 发布于 2023-07-27 / 7403 阅读 / 0 评论 / 28 点赞

1 Docker 简介 1.1 Docker 是什么？ docker 是一个开源的应用容器引擎。 1.2 容器是什么？ 容器是一种轻量级的虚拟化技术，它是一个由应用运行环境、容器基础镜像组成的集合。以 Web 服务 Nginx 为例，如下图所示：Nginx 容器是由 Nginx 主程序、Ngin

1 Docker 简介

1.1 Docker 是什么？

docker 是一个开源的应用容器引擎。

1.2 容器是什么？

容器是一种轻量级的虚拟化技术，它是一个由应用运行环境、容器基础镜像组成的集合。

以 Web 服务 Nginx 为例，如下图所示：Nginx 容器是由 Nginx 主程序、Nginx 运行依赖组件（gcc、pcre、openssl）、CentOS 7 基础镜像组成。（注：CentOS 7 基础镜像并非完整的操作系统镜像，只是操作系统的基础文件和库文件）



1.3 容器与虚拟机的区别



而容器是轻量级的，因为容器只打包了操作系统的基础文件和库文件、还有应用程序及所有的依赖，他的运行速度就如同在系统中创建一个进程一样快，所以启动速度较快。

- **运行性能：**由于虚拟机增加了虚拟化层用于虚拟化硬件，势必会增加一定的开销，所以运行性能有所损失；而容器是直接运行在物理操作系统上的，他本身与系统上其他进程并没有太大区别，所以运行性能是接近原生的。
- **磁盘占用：**虚拟机是一个完整的操作系统，是 GB 级别的，而容器只包含了一些系统启动的必要组件和程序依赖，是 MB 级别的。
- **数量：**运行一个操作系统的开销较大，运行一个进程的开销较小，同样的服务器资源可以运行更多的容器。
- **隔离性：**虚拟机是一个完整的操作系统级别的隔离，要比容器好很多；容器是进程级别的隔离，隔离的不彻底，因为多个容器之间使用的是同一个宿主机的操作系统内核。
- **封装速度：**虚拟机封装会包含操作系统，封装速度比较慢，容器只封装操作系统的基础文件和库文件、应用程序、依赖，封装速度较快。

1.4 Docker 和容器的关系

容器是一种虚拟化技术，docker 是实现容器的一种工具，我们称它为容器引擎；

可以驱动容器的引擎还有 podman、containerd 等，docker 是目前市面上应用范围最广的一种容器引擎。

2 安装 Docker

在 CentOS 7 系统上安装 docker

```
yum install -y yum-utils device-mapper-persistent-data lv
yum-config-manager --add-repo http://mirrors.aliyun.com/d
yum -y install docker-ce && \
mkdir -p /etc/docker
tee /etc/docker/daemon.json <<- 'EOF'
{
  "registry-mirrors": ["https://j6o4qc2l.mirror.aliyuncs.
}
EOF
systemctl daemon-reload
```

3 使用 Docker 启动一个容器

执行以下命令，启动一个 Nginx 容器

```
docker run -d --name nginx_test nginx
```

注释:

```
-d                #后台运行
--name           #自定义容器名称
nginx            #容器镜像
```

使用 `docker ps` 可以看到有一个名为 `nginx_test` 的容器在运行;

```
[root@BI-test ~]# docker run -d --name nginx_test nginx
88ea39f9ca2e3c77ad18400e6fd7afae622f0f8103242ac1143d0550a7f8c8b
[root@BI-test ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS    NAMES
88ea39f9ca2e   nginx     "/bin/sh -c '/opt/ng-'" 5 seconds ago Up 4 seconds    nginx_test
```

进入容器查看服务

```
docker exec -it nginx_test bash
```

注释:

```
-it             #打开终端交互 (进入容器操作)
nginx_test     #容器名称
bash           #执行容器使用的 shell, bash 或 sh。
```

执行命令后, shell 端的主机名变成了随机字段串, 说明进入到了容器内部;

```
[root@BI-test ~]#
[root@BI-test ~]# docker exec -it nginx_test bash
[root@88ea39f9ca2e opt]#
[root@88ea39f9ca2e opt]#
```

在 Nginx 容器中可以看出 Nginx 的服务和端口都是正常的

```
[root@BI-test ~]# docker exec -it nginx_test bash
[root@88ea39f9ca2e opt]# netstat -lnt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
[root@88ea39f9ca2e opt]# ps -ef|grep nginx
root      1      0 09:50 ?        00:00:00 /bin/sh -c /opt/nginx/sbin/nginx && tail -f /dev/null
root      8      1 09:50 ?        00:00:00 nginx: master process /opt/nginx/sbin/nginx
nobody    10     8 09:50 ?        00:00:00 nginx: worker process
root     45    29 09:50 pts/0    00:00:00 grep --color=auto nginx
```

键入 `\exit` 回到宿主机, 执行 `\netstat -lnt` 查看宿主的端口;

如下图所示, 在宿主机并没有 Nginx 的端口, 这是因为容器中的端口并没有映射到宿主主机上, 所以在宿主机无法访问到 Nginx 的服务。

```
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN
tcp6       0      0 :::22                   :::*                     LISTEN
tcp6       0      0 :::1:25                  :::*                     LISTEN
[root@BI-test ~]#
```

3.1 如何访问容器服务

我们在运行容器时使用 `-p` 参数将容器端口映射到宿主
机端口

```
docker run -d --name nginx_test -p 8080:80 nginx
```

注释:

`-p 8080:80` # 8080 代表宿主机端口, 80 代表容器端口

容器运行后, 在宿主机执行 `netstat -lnt` 可以看到,
宿主机已经监听了 8080 端口;

```
[root@BI-test ~]# docker run -d --name nginx_test -p 8080:80 nginx
bb155c6c8d7ae4264c9a8a417a2a9855ab7a1c110e0881b975f66d7b078cc89f
[root@BI-test ~]#
[root@BI-test ~]# netstat -lnt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN
tcp6       0      0 :::8080                  :::*                     LISTEN
tcp6       0      0 :::22                    :::*                     LISTEN
tcp6       0      0 :::1:25                  :::*                     LISTEN
[root@BI-test ~]#
```

查询服务器的 IP, 使用 `curl` 命令访问 IP 地址+端口,
可以返回 Nginx 服务的信息, 说明可以正常访问 Nginx
服务了;

```
[root@BI-test ~]#
[root@BI-test ~]#
[root@BI-test ~]#
[root@BI-test ~]# hostname -I
10.1.13.130 172.17.0.1 192.168.11.254
[root@BI-test ~]#
[root@BI-test ~]# curl 10.1.13.130:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
[root@BI-test ~]#
```

当然, 用浏览器一样可以得到 Nginx 页面信息。

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

3.2 如何将容器内数据持久化存储

我们先来做个测试，测试内容是修改容器内的文件，然后删除容器、重建容器，查看文件是否会保持修改后的状态。

第一步、修改 Nginx 容器的 index.html 查看是否生效

我们尝试通过修改 Nginx 的 index.html 文件来更换页面信息；

```
#先进入 nginx_test 容器中；  
docker exec -it nginx_test bash
```

```
#然后切换到 Nginx 的 html 目录  
cd nginx/html
```

```
#vi 打开 index.html  
vi index.html
```

```
#输入一段文本，并保持退出编辑  
This is the demo page.
```

```
[root@BI-test ~]# docker exec -it nginx_test bash  
[root@bb155c6c8d7a opt]#  
[root@bb155c6c8d7a opt]#  
[root@bb155c6c8d7a opt]# ls  
nginx  
[root@bb155c6c8d7a opt]# cd nginx/html/  
[root@bb155c6c8d7a html]#  
[root@bb155c6c8d7a html]# ls  
50x.html index.html  
[root@bb155c6c8d7a html]#  
[root@bb155c6c8d7a html]# vi index.html  
[root@bb155c6c8d7a html]#  
[root@bb155c6c8d7a html]# cat index.html  
This is the demo page.  
[root@bb155c6c8d7a html]#
```

退出容器，使用 curl 命令访问 Nginx 服务

```
curl 10.1.13.130:8080
```

可以返回我们更换的文本内容，index.html 修改完成。

```
[root@BI-test ~]#  
[root@BI-test ~]# curl 10.1.13.130:8080  
This is the demo page.  
[root@BI-test ~]#  
[root@BI-test ~]#  
[root@BI-test ~]#
```

第二步、删除 Nginx 容器，然后重新创建 Nginx，观察 index.html 内容是否保持修改后状态

```
#删除 nginx_test 容器  
docker rm -f nginx_test  
  
#重新创建 nginx_test  
docker run -d --name nginx_test -p 8080:80 nginx
```

```
[root@BI-test ~]# docker rm -f nginx_test  
nginx_test  
[root@BI-test ~]#  
[root@BI-test ~]# docker run -d --name nginx_test -p 8080:80 nginx  
63bcd5723e824e3bcb9d33b059806321f56c36386196e2365ab5ac506d87a4  
[root@BI-test ~]#
```

完成重建后，使用 curl 命令访问 nginx 服务

```
curl 10.1.13.130:8080
```

```
[root@BI-test ~]# curl 10.1.13.130:8080  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to nginx!</title>  
<style>  
html { color:scheme: light dark; }  
body { width: 35em; margin: 0 auto;  
font-family: Tahoma, Verdana, Arial, sans-serif; }  
</style>  
</head>  
<body>  
<h1>Welcome to nginx!</h1>  
<p>If you see this page, the nginx web server is successfully installed and  
working. Further configuration is required.</p>  
  
<p>For online documentation and support please refer to  
<a href="http://nginx.org/">nginx.org</a>.<br/>  
Commercial support is available at  
<a href="http://nginx.com/">nginx.com</a>.</p>  
  
<p><em>Thank you for using nginx.</em></p>  
</body>  
</html>  
[root@BI-test ~]#
```

可见，curl 返回了 Nginx 默认页面信息，并不是我们修改的内容，这是因为我们删除了容器，容器内的所有文件都一同删除了。如果想在删除容器时，保持指定文件或目录不被删除，该怎么做呢？

将宿主机的目录映射到容器目录，我们称这个操作为：持久化存储

在创建容器时使用 -v 参数将宿主机目录映射到容器目录

```
docker run -d --name nginx_test -p 8080:80 -v /data/nginx
```

注释：

```
-v /data/nginx:/opt/nginx/html #/data/nginx 是宿主机目录，
```

然后我们用同样的方法，在 Nginx 容器中新建一个 index.html，内容自定义，然后删除容器、重建容器；

```
[root@BI-test ~]#
[root@BI-test ~]#
[root@BI-test ~]# docker exec -it nginx_test bash
[root@81429cf60caa opt]#
[root@81429cf60caa opt]# vi /opt/nginx/html/index.html
[root@81429cf60caa opt]# 进入容器新建 index.html 内容
[root@81429cf60caa opt]# cat /opt/nginx/html/index.html
this is the persist page.
[root@81429cf60caa opt]#
[root@81429cf60caa opt]# exit
exit
[root@BI-test ~]#
[root@BI-test ~]# curl 10.1.13.130:8080 访问 Nginx 返回文本内容
this is the persist page.
[root@BI-test ~]#
[root@BI-test ~]#
[root@BI-test ~]# docker rm -f nginx_test 删除重建 Nginx
nginx_test
[root@BI-test ~]#
[root@BI-test ~]# docker run -d --name nginx_test -p 8080:80 -v /data/nginx:/opt/nginx/html nginx
f5a703801df5fd55b85d15e4f35b28cc835f1769bc4cf5ce94050e15b4059d
[root@BI-test ~]#
[root@BI-test ~]# curl 10.1.13.130:8080 重新访问 Nginx 仍然是最初修改的内容
this is the persist page.
[root@BI-test ~]#
[root@BI-test ~]#
[root@BI-test ~]#
[root@BI-test ~]#
```

可以发现，index.html 文件会保持自定义的内容；这是因为 index.html 创建到了宿主机的目录 /data/nginx 中，宿主机的目录 /data/nginx 映射到了容器中的 /opt/nginx/html 目录，即使删除容器，宿主机的文件也不会删除，再次创建容器仍然映射原目录，使得文件持久化存储。

4 自己构建一个镜像

我们在创建运行容器时使用命令：``docker run -d nginx_test nginx``

命令最后的“nginx”是 Docker 官方仓库提供的 Nginx 镜像，我们也可以自己构建一个镜像来使用。

构建镜像需要使用 dockerfile 文件，我们以 Nginx 为例，编写一个 Nginx 镜像的 dockerfile：

```
vi dockerfile

FROM centos:7.9.2009

WORKDIR /opt

ADD nginx-1.24.0.tar.gz /opt

RUN yum install -y nc net-tools gcc-c++ pcre pcre-devel z
yum clean all && \
rm -rf /tmp/* rm -rf /var/cache/yum/* && \
cd /opt/nginx-1.24.0 && \
./configure --user=nobody --group=nobody --prefix=/opt/ng
make && make install && \
rm -rf /opt/nginx-1.24.0

CMD /opt/nginx/sbin/nginx && tail -f /dev/null
```

“注释：

目录, /opt/nginx, /opt/nginx表示容器目录

--name #设置容器名称

nginx #容器镜像”

查看正在运行的容器

```
docker ps
```

“参数:

-a #查看运行的所有容器, 包括运行状态和停止状态的容器”

启动、停止、重启容器

```
docker start nginx_test #启动 Nginx 容器
```

```
docker stop nginx_test #停止 Nginx 容器
```

```
docker restart nginx_test #重启 Nginx 容器
```

进入容器

```
docker exec -it nginx_test bash
```

“参数:

-it #打开终端交互 (进入容器操作) ”

删除运行的容器

```
docker rm -f nginx_test
```

查看容器镜像

```
docker images
```

删除容器镜像

```
docker rmi nginx:latest
```

```
docker rmi d6454d54b3d9 (IMAGE ID)
```

下载容器镜像

```
docker search nginx
```

```
docker pull nginx
```

镜像导出为文件

```
docker save nginx:latest nginx.tar
```

编译镜像

```
docker build -t nginx:v1 .
```

6 容器编排 docker-compose

6.1 docker-copose 介绍

- docker-compose 是一个容器编排工具（自动化部署、管理）；
- 它用来在单台 Linux 服务器上运行多个 Docker 容器；
- docker-compose 使用YAML文件来配置所有需要运行的 Docker 容器，该 YAML 文件的默认名称为 docker-compose.yml

6.2 docker-compose 安装

```
curl -L "https://github.com/docker/compose/releases/download  
chmod +x /usr/local/bin/docker-compose  
ln -s /usr/local/bin/docker-compose /usr/bin/docker-compo  
docker-compose version
```

6.3 使用 docker-compose 启动一个容器

以下是 docker-compose.yml 的内容

```
version: '2.1'  
services:  
  nginx:  
    image: nginx:latest  
    container_name: nginx_test  
    ports:  
      - 8080:80  
    volumes:  
      - /opt/nginx:/opt/nginx/html
```

“参数：

compose 文件格式的版本，恒定为 2.1

services 标签下可以定义多个类似 *nginx* 这样的服务

container_name 服务定义，*nginx_test* 是容器的名称

image *nginx*容器所使用的镜像

ports 定义端口映射，本例将容器内的 80 端口映射到宿主机的 8080 端口



启动容器

在 docker-compose 所在目录执行

```
docker compose up -d
```

“参数:

up 表示启动

-d 表示后台运行

-f 指定 *docker-compose* 文件位置 *docker-compose -f /root/docker-compose/docker-compose.yml up -d*”

6.4 Dockerdocker-compose 命令总结

在 docker-compose 所在目录执行

启动容器

```
docker-compose up -d
```

停止容器

```
docker-compose down
```

重启容器

```
docker-compose restart
```

重载 docker-compose.yml

```
docker-compose up --force-recreate -d
```

6.5 docker-compose 创建多个容器

使用 docker-compose 启动 Nginx 和 Redis 两个容器:

docker-compose.yml 内容如下:

```
version: '2.1'
services:

  nginx:
    image: nginx:latest
    container_name: nginx_host1
    ports:
      - 8081:80
```

[至顶](#)

返回
首页 /
返回
上一页

```
- host1-network
```

```
redis:
```

```
image: redis:latest
```

```
container_name: redis_host1
```

```
ports:
```

```
- 63790:6379
```

```
networks:
```

```
- host1-network
```

```
networks:
```

```
host1-network:
```

```
driver: bridge
```

```
ipam:
```

```
driver: default
```

```
config:
```

```
- subnet: 192.168.11.0/24
```

```
gateway: 192.168.11.254
```

“参数:

networks 定义容器网络, *host1-network* 为定义的网络名称,

config 网络配置, *subnet* 代表网段, *gateway* 代表网关。”

执行创建命令

```
docker-compose up -d
```

可以看到成功创建了 Nginx 和 Redis 两个容器

```
[root@BI-test docker-compose]# docker-compose up -d 创建容器
[*] Building 0.0s (0/0)
[*] Running 2/2
  Container nginx_host1 Started 0.3s
  Container redis_host1 Started 0.3s
[root@BI-test docker-compose]# docker ps 查看容器
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS                               NAMES
c9278c9db05da  nginx:latest  "/bin/sh -c 'opt/nginx'" 31 seconds ago  Up 30 seconds  0.0.0.0:8081->80/tcp, :::8081->80/tcp  nginx_host1
fcfc3fe02d02  redis:latest  "docker-entrypoint.s"      31 seconds ago  Up 30 seconds  0.0.0.0:63790->6379/tcp, :::63790->6379/tcp  redis_host1
[root@BI-test docker-compose]#
```

可以进入 nginx_host1 容器查看一下端口和 IP

```
[root@BI-test docker-compose]#
[root@BI-test docker-compose]# docker exec -it nginx_host1 bash 进入 nginx_host1 容器
[root@c9278c9db05da opt]#
[root@c9278c9db05da opt]# ifconfig 查看 IP 地址
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.11.2 netmask 255.255.255.0 broadcast 192.168.11.255
    ether 02:42:c0:a8:0b:02 txqueuelen 0 (Ethernet)
    RX packets 8 bytes 656 (656.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

[root@c9278c9db05da opt]#
[root@c9278c9db05da opt]# telnet redis_host1 6379
bash: telnet: command not found
[root@c9278c9db05da opt]#
[root@c9278c9db05da opt]# nc -zv redis_host1 6379 访问 Redis 容器服务
Ncat: Version 7.50 ( https://nmap.org/ncat )
Ncat: Connected to 192.168.11.116379.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.
[root@c9278c9db05da opt]#
[root@c9278c9db05da opt]#
```

1 Docker
简介

1.1

Docker
是什么?

1.2 容器
是什么?

1.3 容器
与虚拟机
的区别

1.4
Docker
和容器
的关系

2 安装
Docker

3 使用
Docker 启
动一个容器

3.1 如何
访问容
器服务

3.2 如何
将容器
内数据
持久化
存储

4 自己构建
一个镜像

5 总结
Docker 命
令

6 容器编排
docker-
compose

6.1
docker-
compose
介绍

6.2
docker-
compose



如果访问 Redis 可以直接使用docker-compose 定义的 redis_host1 容器名访问即可。

以上是 Docker 的入门教程，欢迎大家留言评论交流！

上一篇: 【模板应用】通过 DataEase (v1.1... 下一篇: 【使用

是否对你有帮助? 3 4



飞致云知识库

又好又快解决你的问题

更多资源

关于我们
飞致云开源社区
飞致云学堂
飞致云技术博客

关注我们



docker-compose
启动一个容器
6.4
Dockerdoc
compose 命令
总结
6.5
docker-compose
创建多个容器

状图展示项目进

分类目录

JumpServer
DataEase 21
MeterSphere
MaxKB 7
CloudExplorer
Lite

标签更多

使用场景 105
MacOS 0
Razor 1
扩展类 27
模板应用 8
beanshell 3
JSON 1
接口测试 11
功能介绍 83
常见问题 113

